B.Sc. (Hons) in IT – Field of Specialization Interactive Media

2026 – Year 04 – Semester 01

SE4031 - Games Development

Neon Astro

Student Name	Amarawardhane.A.S
IT Number	IT22237804
Module	SE4031 - Games Development
Submission	Assignment 01 (Individual) - 2D Interactive Game

1. Game Title

Neon Astro

2. Summary of the Game Concept

Neon Astro is a 2D side-scrolling space survival game where the player pilots a high-tech Space Ship through a dangerous asteroid belt. The goal is to survive by dodging or destroying Asteroids using sonic pulses while collecting Battery packs to maintain hull integrity. The mission is completed once the player achieves a score of 100 points.

3. Description of How Core Features Are Implemented

- **Health System (Hull Integrity):** The Space Ship has a maximum health of 100. This is displayed on the UI via a dynamic text system that changes color (from green to red) as health decreases. Colliding with an Asteroid reduces health by 20 points. If health reaches zero, the ShowGameOver() function is triggered.

- **Score System:** The score is displayed as numerical digits on the HUD. Points are earned primarily by destroying **Asteroids** with projectiles (10 points per destruction). The ScoreAndMenu script updates the UI in real-time and checks if the score has reached the winning threshold of 100.

- **Health Packs (Battery):** Health is restored using Battery packs that spawn randomly. These are trigger-based objects; when the Space Ship's collider overlaps with a Battery, 20 health points are restored (up to the 100 max). If not collected, they are automatically destroyed after passing the screen to save memory.

- **Projectile Attacks:** The player can fire projectiles using the Left Mouse Button. The PlayerLogic script handles the instantiation of the projectile prefab, while a separate Projectile script controls its forward movement. It also includes a safety check to prevent

firing when clicking on UI buttons.

- **Enemy Logic (Asteroids):** The primary hazards are Asteroids. They behave as damaging obstacles that move from right to left. They can be destroyed by projectiles, which adds to the player's score. The EnemyAndItems script manages their movement, collision damage, and self-destruction logic.

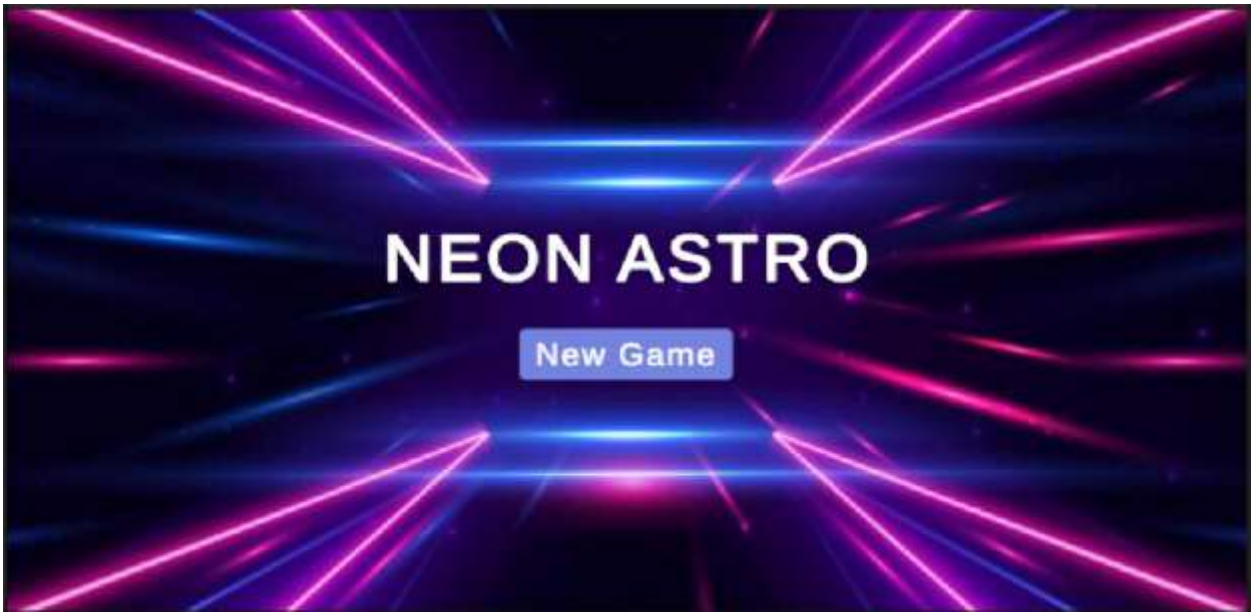
- **Dynamic Speed & Spawning:** The game features a dynamic difficulty curve implemented in the Spawner script:

    - **Speed:** The movement speed of obstacles increases by 0.05 for every 1 point earned, capped at a maximum speed of 15.
    - **Spawn Rate:** The interval between spawns decreases as the score goes up, making the game more intense as the player nears 100 points.

- **Menus and End States:**

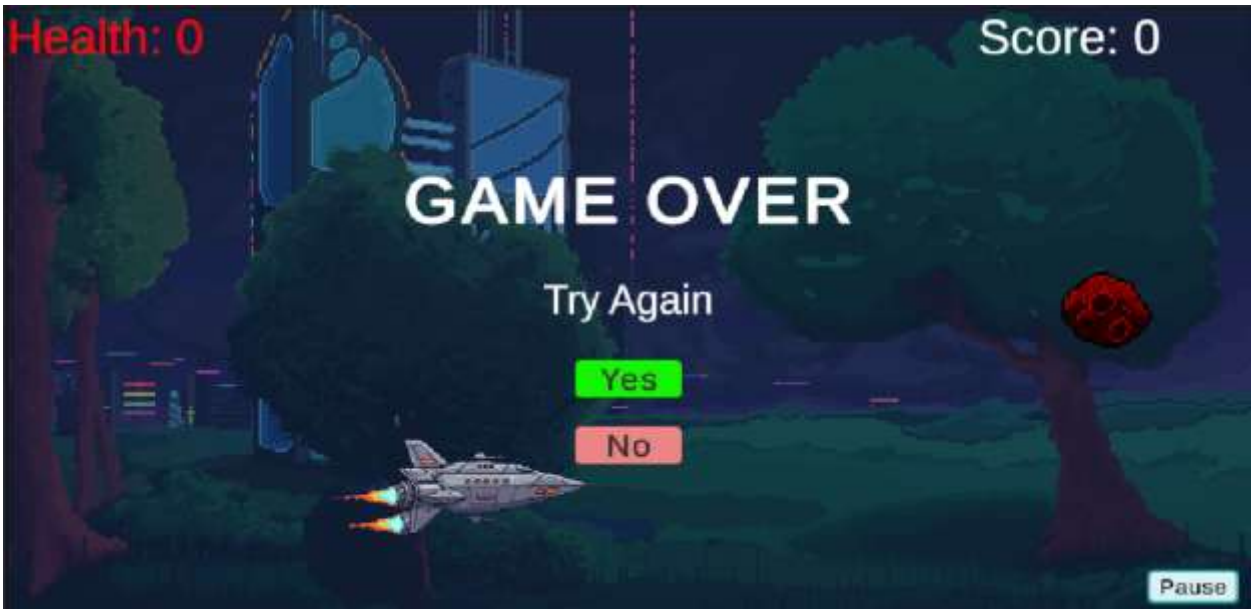
    - **Start Menu:** Managed by MainMenuController to load the game scene.
    - **Pause/Resume:** Allows players to pause the game and music using Time.timeScale = 0.
    - **Game Over/Win:** Specific panels appear based on whether health reaches 0 or score reaches 100, accompanied by the StopMusic() command.

4. Screenshots of Gameplay



Health: 100
Score: 0
Pause

Health: 80
Score: 100
MISSION COMPLETE
Main Menu
Pause



5. Control Guide

Action	Input (Keyboard/Mouse)
Move Up	W Key / Up Arrow Key
Move Down	S Key / Down Arrow Key
Play Button	Start the game from the title screen
Restart Button	Restart after Game Over or Win
Pause Game	UI Pause Button
Menu Button	Direct to title screen
Navigation	Mouse Left-Click (for UI Buttons)

6. Description of the Creative Feature

- **Dynamic Health Gradient System:** Unlike static health bars, Neon Astro features a real-time color gradient system. Using the Color.Lerp function, the health text color constantly updates along a smooth gradient from Green to Red based on the exact percentage of current health. This provides immediate and intuitive visual feedback to the player about their ship's status.

- **Adaptive Environment Difficulty:** The game implements a "Score-Synced Difficulty Curve." The speed of the asteroids and the rate at which they spawn are directly tied to the player's real-time score. This creates a seamless transition from a relaxing space flight to a high-intensity survival challenge without needing separate level loads.

- **Visual Collision Indicators:** When the player collects a Battery Pack or hits an **Asteroid**, the object is instantly destroyed and the UI updates immediately, providing a clear "hit-marker" effect. This ensures the player knows exactly when a resource has been successfully gathered or damage has been taken.

- **Integrated Audio Feedback State:** The background music (BGM) is synchronized with the game’s logic states. It doesn't just loop; it responds to the "Mission Complete" and "Game Over" triggers by stopping instantly, providing a powerful psychological sense of closure or failure to the player.

7. Credits (External Assets & Tools)

Item	Details
Engine	Unity 2D (Version 2022.3 LTS)
Code Editor	Visual Studio Code / C#
UI Components	TextMeshPro (Unity Technologies)
Audio	Royalty-free Synthwave track sourced via No Copyright Music (YouTube Library).

8. Gameplay Video Link

Video Link: IT22237804-Demo Video.mp4

OneDrive link of the game: IT22237804_Assignment_01.zip